will result in that request failing. If we're able to extract this code into an async job, then we can retry it after the system has had a chance to recover.

Do we need to control this code's concurrency?

Elixir will let you run a lot of requests simultaneously. This is both a good and bad thing. Often, we need to throttle the amount of concurrency that can happen with a given piece of code. If we use an async job system, we have full control over the amount of concurrency.

Does this piece of code need to execute immediately, or can it wait a bit?

If the code must run as part of the request, then it will be difficult to make async. But if it's acceptable to happen after the fact, then we're free to extract it into an async job.

Does the job system have features that simplify my development process?

Job systems often offer more than just executing units of work. They provide features like uniqueness, workflow management, batching, and more. You can tap into these features to save massive amounts of time on certain problems.

It helps to know what your job system can do so you know what is a good fit for it!

Now that we've established when we need a job system, here's how to pick a good one.

Requirements of a Good Job System

Not all job systems are built the same. Some are simple and seek to solve a single problem—execute async jobs. Other job systems provide robust features that make them quite powerful by comparison. Truly great job systems will provide the advanced features you need, but they make basic operation so easy that anyone can use them. Sidekiq in Ruby is a great example of a great job system that pulls this off well.

Here are some things to consider when looking at a job system:

Is it audited?

We need to know whether a job failed or succeeded, its progress, and so on. We should be in control of how long we hold this audit for.

Is it modular?

Ideally, we won't need to update the code powering our job system. But it's helpful to be able to implement our own logic or feature for any uncommon use cases we may have.